Context_Management

Table of Contents

- 1. Overview
  - 1.1 Architecture Layers Overview
  - 1.2 System Design Principles
- 2. Token Counting System
  - 2.1 Exact Counting vs. Estimation
  - 2.2 Rough Estimation (When Exact Data Is Unavailable)
  - 2.3 Hybrid Counting (Standard Function)
  - 2.4 Special Content Type Estimation
  - 2.5 Haiku-Assisted Counting
  - 2.6 Bedrock Special Handling
- 3. Context Window Management
  - 3.1 Core Constants
  - 3.2 Model Context Window Calculation
  - 3.3 Output Token Management
  - 3.4 Threshold Percentage System
  - 3.5 Blocking Limit
- 4. Microcompact
  - 4.1 Two Trigger Modes
  - 4.2 Microcompact Decision Flow
  - 4.3 Warning Suppression in Microcompact
  - 4.4 API Layer Injection
- 5. Snip History Pruning
  - 5.1 How It Works
  - 5.2 Relationship with Compaction
  - 5.3 Snip Preservation Strategy
  - 5.4 Snip Trigger Conditions
- 6. API Layer Context Management
  - 6.1 Strategy Types
  - 6.2 Usage Scenarios
- 7. Full Compaction
  - 7.1 Trigger Conditions
  - 7.2 Complete Execution Flow
  - 7.3 Compaction Summary Prompt Design
  - 7.4 Image Stripping Processing
  - 7.5 File Recovery After Compaction
  - 7.6 Skill Recovery
  - 7.7 PTL Retry Mechanism
  - 7.8 Prioritizing Fork Sub-Agent Path During Compaction
  - 7.9 Direct Streaming Path (Fallback)
- 8. Session Memory
  - 8.1 Core Difference from Compaction
  - 8.2 Trigger Conditions

- 8.3 Sub-Agent Execution Flow
- 8.4 Session Memory Template Details
- 8.5 Token Limits
- 8.6 Custom Template Support
- 8.7 Tool Usage Restrictions
- 8.8 Mutex with Main Agent
- 8.9 Usage of Session Memory
- 9. Session Memory Compaction
  - 9.1 Comparison with Traditional Compaction
  - 9.2 Preserved Message Calculation
  - 9.3 Feature Availability Check
  - 9.4 Execution Flow
  - 9.5 Compaction Boundary Metadata
- 10. Auto-Compact
  - 10.1 Threshold System Explained
  - 10.2 Warning States
  - 10.3 Compaction Priority
  - 10.4 Circuit Breaker Explained
  - 10.5 Disable Control Matrix
  - 10.6 Special Case of Marble Origami
- 11. Context Collapse
  - 11.1 Core Difference from Compaction
  - 11.2 Design Principles
  - 11.3 Mutual Exclusion with Auto-Compact
  - 11.4 Implementation Location
  - 11.5 Objects Being Collapsed
  - 11.6 Two-Phase Collapse Algorithm
  - 11.7 Relationship with Other Mechanisms
- 12. Context Analysis
  - 12.1 Use Cases
  - 12.2 Statistical Dimensions
  - 12.3 Duplicate Read Detection Algorithm
  - 12.4 Statsig Metric Mapping
  - 12.5 Detecting Optimizable Waste Points
  - 12.6 Summary
- 13. Post-Compaction Cleanup
  - 13.1 Cleanup Contents Explained
  - 13.2 Key Items NOT Cleared
  - 13.3 Main Thread vs. Sub-Agent Differentiation
- 14. Context Loading and Caching
  - 14.1 System Context
  - 14.2 Session Storage
- 15. Persistent Memory Extraction
  - 15.1 Difference from Session Memory
  - 15.2 Flow

    - 15.3 Storage Structure and Content
    - 15.4 Sub-Agent Execution
    - 15.5 Mutex Protection
    - 15.6 Exit Wait
- 16. History Management
    - 16.1 Key Constants
    - 16.2 Main Functions
- 17. Message Normalization and Boundary Management
    - 17.1 Message Normalization
    - 17.2 Compaction Boundary Messages
    - 17.3 Microcompact Boundary Messages
    - 17.4 Finding Compaction Boundaries
- 18. Key Design Decisions
    - 18.1 Why Fork Sub-Agent for Summary Execution?
    - 18.2 Why Microcompact at API Layer Instead of Direct Message Modification?
    - 18.3 Why Use GrowthBook for Dynamic Configuration?
- 19. Global Data Flow Overview
    - 19.1 Complete Request Processing Flow
    - 19.2 Compaction Message Construction Flow
    - 19.3 Token Consumption by Context Layer

---

1. Overview

Claude Code's context management system is one of the most core and complex subsystems in the entire project. It is composed of multiple layers of mechanisms, ranging from low-level token counting, to mid-level microcompact (tool result pruning), to high-level full compaction (conversation history summarization), and finally session-level and persistent-level memory management. The entire system revolves around one core goal: **maximizing effective information within a limited context window**.

1.1 Architecture Layers Overview

```
┌──────────────────────────────────────────────────────────┐
│  Layer 5: Persistent Memory Extraction (extractMemories) │
│            → Persists across sessions to ~/.claude/projects/ │
├──────────────────────────────────────────────────────────┤
│  Layer 4: Session Memory                                 │
│            → Markdown notes for the current session      │
├──────────────────────────────────────────────────────────┤
│  Layer 3: Full Compaction                                │
│            → AI-summarized history (model invocation)   │
├──────────────────────────────────────────────────────────┤
│  Layer 2: History Pruning (Snip)                         │
│            → Selective discarding (by time/token)        │
├──────────────────────────────────────────────────────────┤
│  Layer 1: Microcompact                                   │
│            → Clean old tool results, keep hint cache     │
├──────────────────────────────────────────────────────────┤
│  Layer 0: Token Counting & Threshold Detection           │
│            → Mathematical foundation for all decisions   │
└──────────────────────────────────────────────────────────┘
```

1.2 System Design Principles

- **Layered Management**: Each layer handles problems at different granularities (Token-level → Tool-level → Message-level → Session-level)
- **Conservative Triggers**: All auto mechanisms have circuit breakers to prevent wasted API calls from repeated failures
- **Zero Information Loss Intent**: Compress history through structured summarization rather than simple discarding
- **Prompt Cache Friendly**: Compaction operations prioritize reusing existing API prompt caches
- **Progressive Optimization**: Prefer lightweight mechanisms first (MC → Snip → Compaction), with heavy mechanisms as fallback

---

2. Token Counting System

Token counting is the foundation of the entire context management system—all compaction decisions are based on token counts.

2.1 Exact Counting vs. Estimation

Claude Code uses two token counting strategies, each suited for different scenarios:

Exact Counting (After API Call)

```
// utils/tokens.ts
getTokenUsage(message: Message): Usage | undefined

// Extracts from the usage field of assistant messages
// Conditions: message has usage field + not a synthetic message + not SYNTHETIC_MODEL
// Does not return progress/attachment/system type messages
```

The `Usage` object contains:

```
{
  input_tokens: number,
  output_tokens: number,
  cache_creation_input_tokens?: number,  // Cache creation cost
  cache_read_input_tokens?: number,      // Cache read savings
}
```

Exact Total Context Window Token Count

```
// utils/tokens.ts
getTokenCountFromUsage(usage: Usage): number
// = input_tokens + cache_creation_input_tokens + cache_read_input_tokens + output_tokens
// This number represents the full context size at the time of this API call
```

Getting Exact Count from Historical Messages

```
// utils/tokens.ts
tokenCountFromLastAPIResponse(messages: Message[]): number
// Traverse messages array backward, find the first assistant message with usage
// Return getTokenCountFromUsage for that message
// Used for token count comparison before and after compaction
```

Final Window Token for task_budget

```
// utils/tokens.ts
finalContextTokensFromLastResponse(messages: Message[]): number
// Gets final window size from usage.iterations[-1]
// (Used for server-side tool loop scenarios)
// Formula: input_tokens + output_tokens (excludes cache tokens)
```

2.2 Rough Estimation (When Exact Data Is Unavailable)

```
// services/tokenEstimation.ts
roughTokenCountEstimation(content: string, bytesPerToken = 4): number
// Simplest estimation: character count ÷ 4
// Suitable for plain text content
```

Ratio adjustment by file type:

```
// JSON/JSONL/JSONC files have byte/token ratio close to 2 (due to many single-character symbols)
// Other files use default 4
bytesPerTokenForFileType(fileExtension: string): number {
  switch (fileExtension) {
    case 'json': case 'jsonl': case 'jsonc': return 2
    default: return 4
  }
}
```

2.3 Hybrid Counting (Standard Function)

```
// utils/tokens.ts
tokenCountWithEstimation(messages: readonly Message[]): number
```

This is the **most important counting function**, serving as the basis for auto-compaction threshold decisions. The logic is:

> **PS**: Use exact calculation where possible; only fall back to rough estimation for the rest.

```
1. Traverse backward from the last message
2. Find the first assistant message with exact usage data
3. If found:
   a. Traverse forward, including all split messages with the same message.id in estimation
      (When the model makes parallel tool calls in one response, streaming code emits separate
      assistant message records for each content block, sharing message.id and usage,
      with tool_results inserted immediately between tool_uses)
   b. Exact usage + rough estimation for subsequent messages
4. If not found: use rough estimation for all
```

Handling of parallel tool calls: When the model calls multiple tools in parallel within one response, the streaming code emits separate assistant message records for each content block (sharing `message.id` and `usage`), inserting tool_results immediately between tool_uses. The message array structure is:

```
[..., assistant(id=A, usage), user(tool_result_A), assistant(id=A, usage),
 user(tool_result_B), assistant(id=A, usage), user(tool_result_C), ...]
```

If one only stops at the last assistant record, all intermediate tool_results will be missed, leading to underestimation. The backtracking logic in `tokenCountWithEstimation` includes all records sharing the same `message.id` in the estimation.

Note:

```
- **Blind spot in conventional counting logic:** Many telemetry systems, when calculating the token
  consumption of a conversation round or assembling historical messages, habitually "find the last
  assistant message" as the endpoint of the current turn.

- **Why underestimation happens:** If a system only grabs the last `assistant(id=A, usage)` to
settle
  the count, it will ignore all previous `assistant` records that also belong to `id=A`, and will
  miss the huge text embedded in between — namely those `tool_result`s (tool return results
  typically contain many tokens). This causes the system's calculated token count to be much lower
  than the actual consumption (underestimation).
```

2.4 Special Content Type Estimation

```
// services/tokenEstimation.ts
roughTokenCountEstimationForBlock(block): number

// Text blocks: character count ÷ 4
if (block.type === 'text')
  return roughTokenCountEstimation(block.text)

// Image/document blocks: fixed 2000 tokens
// Reason: Claude platform charges for images as (width × height) / 750
// Upper limit 2000×2000 = 5333 tokens, conservative value 2000 is used
if (block.type === 'image' || block.type === 'document')
  return 2000  // IMAGE_MAX_TOKEN_SIZE

// tool_use: tool name + JSON-stringified input parameters
// Note: input here can be arbitrarily large (bash commands, file contents, etc.)
// Stringify once directly for estimation, because the API will re-serialize anyway
if (block.type === 'tool_use')
  return roughTokenCountEstimation(
    block.name + JSON.stringify(block.input ?? {})
  )

// tool_result: recursively estimate its content
if (block.type === 'tool_result')
  return roughTokenCountEstimationForContent(block.content)

// thinking blocks: only count thinking text, excluding JSON wrapper and signature
if (block.type === 'thinking')
  return roughTokenCountEstimation(block.thinking)

// Edited thinking: estimate the data field
if (block.type === 'redacted_thinking')
  return roughTokenCountEstimation(block.data)

// All other block types (server_tool_use, web_search_tool_result, etc.)
// Unified stringify then estimate
return roughTokenCountEstimation(JSON.stringify(block))
```

2.5 Haiku-Assisted Counting

```
// services/tokenEstimation.ts
countTokensViaHaikuFallback(messages, tools): Promise<number>
```

When exact usage is unavailable (e.g., messages after cross-session recovery), Claude Code uses Haiku (the fastest model) to estimate token count:

```
const model = isVertexGlobalEndpoint || isBedrockWithThinking || isVertexWithThinking
  ? getDefaultSonnetModel()   // Vertex global endpoint or Bedrock+thinking: use Sonnet
  : getSmallFastModel()         // Otherwise use Haiku 4.5 (supports thinking)
// Calls anthropic.beta.messages.create (max_tokens=1 or 2048)
// Returns input_tokens + cache_creation + cache_read
```

2.6 Bedrock Special Handling

```
// services/tokenEstimation.ts
countTokensWithBedrock({ model, messages, tools, betas, containsThinking })

// Bedrock's CountTokens API requires base model ID (not inference profile ARN)
// Needs conversion via getInferenceProfileBackingModel()
// Model: bedrock-2023-05-31
```

---

3. Context Window Management

3.1 Core Constants (`utils/context.ts`)

```
MODEL_CONTEXT_WINDOW_DEFAULT = 200_000    // Default context window: 200K tokens
COMPACT_MAX_OUTPUT_TOKENS = 20_000       // Max output for compaction model call
ESCALATED_MAX_TOKENS = 64_000            // Escalated mode max output
CAPPED_DEFAULT_MAX_TOKENS = 8_000         // Default capped mode max output
```

3.2 Model Context Window Calculation

```
getContextWindowForModel(model: string, betas?: string[]): number
// Standard models: 200K
// Models supporting 1M (via has1mContext / modelSupports1M): 1,000,000
// Special beta configurations: may be smaller

getEffectiveContextWindowSize(model: string): number
// = getContextWindowForModel(model) - MAX_OUTPUT_TOKENS_FOR_SUMMARY(20K)
// Reserve 20K for model output, actual usable input window
// Supports env var override: CLAUDE_CODE_AUTO_COMPACT_WINDOW=150000
```

3.3 Output Token Management

```
getModelMaxOutputTokens(model: string): number
// Different models have different output limits:
// Sonnet 4.6: 64K
// Sonnet 4.5: 32K
// Haiku: 8K
// ...
```

3.4 Threshold Percentage System

```
Effective context window (effectiveWindow)
  ┌──────────────────────────────────────────┬────────────────┐
  │                                          │                │ ← Output buffer (20K)
```

```
  │  0%         Warning line          93.5%│  100%   100% │
  │     ├────────────────┤├────────────────┤   │              │
  │     WARNING_BUFFER  AUTO_BUFFER        │ Overflow!    │
  │      (20K)         (13K)               │              │
  └──────────────────────────────────────────┴──────────────┘

          ↑
     calculateTokenWarningState(tokenUsage, model)
     Returns:
       percentLeft           = remaining percentage
       isAboveWarningThreshold      // Warning
       isAboveErrorThreshold        // Error
       isAboveAutoCompactThreshold  // Auto-compact
       isAtBlockingLimit            // Blocking
```

3.5 Blocking Limit

```
// Beyond this threshold, block user input (prevent overflow)
isAtBlockingLimit = tokenUsage >= blockingLimit
// blockingLimit = effectiveWindow - MANUAL_COMPACT_BUFFER_TOKENS(3K)
// Approximately effectiveWindow - 3K

// Supports env var override: CLAUDE_CODE_BLOCKING_LIMIT_OVERRIDE
```

---

4. Microcompact

Microcompact is the lightest-weight context management mechanism executed before each API call. It does not summarize; instead, it directly cleans or replaces content in old tool results.

Note: This is actually pruning

4.1 Two Trigger Modes

4.1.1 Time-Based Microcompact

Core idea: Claude API's prompt cache has a 1-hour (60-minute) TTL. When the time since the last assistant message exceeds 60 minutes, **the cache has expired** and the entire prefix will be recalculated. At this point, **proactively cleaning old tool results can reduce the amount of data to be rewritten**.

Note: Whether a cache hit occurs depends not only on whether the prefix is the same, but also on the TTL situation. However, this TTL is calculated by measuring the time difference between two requests.

```
// services/compact/timeBasedMCConfig.ts
const TIME_BASED_MC_CONFIG_DEFAULTS = {
  enabled: false,               // Off by default, enabled via GrowthBook 'tengu_slate_heron'
  gapThresholdMinutes: 60,    // 60 minutes (aligned with server cache TTL)
  keepRecent: 5,               // Keep the most recent 5 compactable tool results
}
```

Trigger judgment:

```
evaluateTimeBasedTrigger(messages, querySource): { gapMinutes, config } | null
// Must satisfy:
// 1. config.enabled = true (GrowthBook switch)
// 2. querySource === 'repl_main_thread' (main thread only)
// 3. Has a most recent assistant message
// 4. gap = (now - lastAssistant.timestamp) / 60000 >= config.gapThresholdMinutes
```

Processing logic (`maybeTimeBasedMicrocompact`):

1. `collectCompactableToolIds()` collects tool_use_ids of all **compactable tools** (in order of appearance)
2. Keep the most recent N (`keepRecent`), add the rest to `clearSet` → Pruning protection
3. Traverse messages, replace tool_result content for tools in `clearSet` with `[Old tool result content cleared]`
4. **Key point**: Modifying prompt content breaks server-side cache, so reset Cached MC state and notify cache break detection

Note: This also performs a one-shot pruning at once, which is more reasonable than OpenClaw's approach using a window guard for dynamic balance.

Cleanup scope (`COMPACTABLE_TOOLS`):

```
const COMPACTABLE_TOOLS = new Set([
  'Read',
  'Bash', 'PowerShell',  // ...all SHELL_TOOL_NAMES
  'Grep',
  'Glob',
  'WebSearch',
  'WebFetch',
  'Edit',
  'Write',
])
```

AgentTool, TaskOutputTool and other critical tools are **not** in this list and will not be cleaned.

4.1.2 Cached Microcompact

Core idea: Leveraging the API's `cache_edits` mechanism, by specifying `cache_reference` (referencing existing cached segments) and `cache_edits` (edit operations) in the request, old tool results are deleted while **preserving the validity of the prompt cache**.

```
┌──────────────────────────────────────────────────────────────┐
│  Server-side Cache (Prompt Cache)                            │
│                                                              │
│  [System prompt + Tool definitions + Message prefix]  ← Cached, cache_hit │
│                                                              │
│  [Tool Result A: Read foo.py]           ← cache_edit: delete │
│  [Tool Result B: Bash git status]       ← cache_edit: delete │
│  [Tool Result C: Grep "func"]           ← kept              │
│  [Latest user message]                                       │
└──────────────────────────────────────────────────────────────┘
                              ↓
                     API request only transmits delta
                     (what to delete, not everything)
```

PS:
Actual advantage: cache_edit deletion essentially masks some KV, but the KV cache is not broken.
These contents are fields considered in session modeling; they are not in context (all infra layer directive parameters are in session, not in context).

Enable conditions (`cachedMicrocompactPath` in `microCompact.ts`):

1. Feature flag `CACHED_MICROCOMPACT` is on
2. Model supports cache editing (`isModelSupportedForCacheEditing(model)`)
3. `isMainThreadSource(querySource)` is true (`querySource === 'repl_main_thread'` or starts with this prefix)

State tracking implementation (`cachedMicrocompact.js` , dynamic import):

```
CachedMCState = {
  toolOrder: string[],           // All registered tool_use_ids (in order)
  registeredTools: Set<string>,  // Registered tool_use_ids
  deletedRefs: Set<string>,     // Deleted tool_use_ids
  pinnedEdits: PinnedCacheEdits[],  // Pinned edit operations
}
```

Workflow:

1. **Register**: In each user message, for each new tool_result, call `registerToolResult(tool_use_id)` , grouped by user message
2. **Decide**: Call `getToolResultsToDelete()` , based on GrowthBook-configured thresholds to determine which to delete
3. **Pin**: If edits are detected (tool_result of edit-class tools), pin the edit operations (`pinCacheEdits`), ensuring they are sent in the next request too
4. **Execute**: Create `cache_edits` block, marking tool_use_ids to delete
5. **API Layer Injection**: After `microCompactMessages` returns, the API layer adds `cache_reference` and `cache_edits`

4.2 Microcompact Decision Flow

```
// services/compact/microCompact.ts
microcompactMessages(messages, toolUseContext?, querySource?): MicrocompactResult

// Execution order (short-circuit logic):
async function microcompactMessages(messages, toolUseContext, querySource) {
  clearCompactWarningSuppression()  // Clear warning suppression

  // 1. Time-based MC first (detects cache expired)
  const timeResult = maybeTimeBasedMicrocompact(messages, querySource)
  if (timeResult) return timeResult  // Short-circuit: time-based takes priority

  // 2. Cached MC (must meet conditions)
  if (feature('CACHED_MICROCOMPACT') && isCachedMCEnabled() && isModelSupported() && isMainThread)
{
    return cachedMicrocompactPath(messages, querySource)  // Short-circuit
  }

  // 3. No available MC path → return original messages (fallback)
  return { messages }
}
```

4.3 Warning Suppression in Microcompact

```
// services/compact/compactWarningState.ts
compactWarningStore: Store<boolean>

// Set suppression=true immediately after successful microcompact
// UI layer subscribes to this state via useCompactWarningSuppression() hook
// When suppression=true, "context will be compacted soon" warning is not shown
// Reason: The core issue is that **after microcompact, the client doesn't know how large
// the context actually is**; only the next API response has precise data, so the warning
// is not shown since the exact value is uncertain
```

4.4 API Layer Injection

When constructing the API request (`microCompact.ts` → caller), pending `cache_edits` are retrieved via `consumePendingCacheEdits()` and injected into the request. Simultaneously, `cache_deleted_input_tokens` is extracted from the last assistant message as the baseline for incremental calculation.

---

5. Snip History Pruning (This feature is similar to truncation. It is disabled by default.)

Snip (`HISTORY_SNIP` feature flag) is a mechanism that directly discards early content at the message history level without summarization. Unlike compaction, Snip discards content the model no longer needs to see (e.g., file read results that have already been processed).

5.1 How It Works

```
Message history
  [Msg0] [Msg1] ... [MsgN-2] [MsgN-1] [Latest msg]
      ↑                               ↑
   snip point ← ← ← ← ← ← ← ← ← ← ← keep tail

After Snip:
  [SNIP_MARKER] [MsgN-2] [MsgN-1] [Latest msg]
```

5.2 Relationship with Compaction

- **Can coexist**: Snip and compaction are not mutually exclusive and can trigger simultaneously
- **Execution order**: Snip executes before microcompact (`query.ts:396`)
- **Token release tracking**: `snipTokensFreed` is passed to `autoCompactIfNeeded` , ensuring auto-compaction threshold judgments reflect the space already freed by Snip

```
// In query.ts:
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages
  snipTokensFreed = snipResult.tokensFreed
  if (snipResult.boundaryMessage) {
    yield snipResult.boundaryMessage
  }
}
// ... subsequent microcompact and autocompact use snipTokensFreed
```

5.3 Snip Preservation Strategy

Snip does not discard indiscriminately; it preserves:

1. **Most recent N messages**: Ensures working context is not broken
2. **Messages containing tool_use without paired tool_result**: Prevents orphaned tool_results
3. **Session boundary messages**: Such as compaction boundaries, SessionStart messages, etc.

PS:
Directly modifies messages[], replacing a middle segment with a placeholder marker, then only sends the latter part to the API → equivalent to truncating the head.
SnipModule is a compiled dynamic module loaded lazily via require(), so the exact trigger timing is unknown. From the code structure, it appears to be triggered by quantity (alternatively, users can manually trigger via `/force-snip` command).

5.4 Snip Trigger Conditions

1. Feature Flag switch

```
  // Conditional compilation switch — HISTORY_SNIP enables it when true
  feature('HISTORY_SNIP')
```

2. Trigger Method 1: Automatic check in each Query loop (auto-trigger)

```
User input
    ↓
query.ts:401 — Mandatory path before every API call
    ↓
snipModule!.snipCompactIfNeeded(messagesForQuery)
    ↓
[Condition check] Is snip needed?
    ↓
Possible output: messages + tokensFreed + boundaryMessage
```

Core call site `query.ts:401-410`:

```
if (feature('HISTORY_SNIP')) {
  const snipResult = snipModule!.snipCompactIfNeeded(messagesForQuery)
  messagesForQuery = snipResult.messages   // Trimmed messages
  snipTokensFreed = snipResult.tokensFreed  // Freed token count
  if (snipResult.boundaryMessage) {
    yield snipResult.boundaryMessage        // Emit boundary marker
  }
}
```

Execution order: Executes before microcompact (`query.ts:396` comment states).

3. Trigger Method 2: Model proactively calls snip tool (tool call triggered)

SNIP is exposed to the model as a tool (SnipTool):

```
// tools.ts:123
const SnipTool = feature('HISTORY_SNIP')
  ? require('./tools/SnipTool/SnipTool.js').SnipTool
  : null
```

The model can proactively call the snip tool in any turn to tell Claude Code which history segment to prune. Tool call triggers `snipCompactIfNeeded({ force: true })` (see `QueryEngine.ts:1238` snipReplay logic).

4. Trigger Method 3: Manual command `/snip`

```
// commands.ts:83
const forceSnip = feature('HISTORY_SNIP')
  ? require('./commands/force-snip.js').default
  : null
```

When HISTORY_SNIP is on, users can manually call `/snip` to force-trigger pruning.

5. Nudge Mechanism: Periodic reminders for the model to use snip

```
// attachments.ts:3963
export function getContextEfficiencyAttachment(messages: Message[]): Attachment[] {
  if (!isSnipRuntimeEnabled()) return []
  if (!shouldNudgeForSnips(messages)) return []  // ← Trigger check
  return [{ type: 'context_efficiency' }]         // ← Inject reminder
}
```

---

6. API Layer Context Management

When the model API **supports native context management strategies**, you can declare context management strategies in the `context_management` field of the request, and the server's inference engine automatically executes cleanup operations.

PS: The essence is partially handing over context management decisions to the server, letting the server automatically handle at appropriate times, without the client repeatedly sending `cache_edits` or doing content replacement.

6.1 Strategy Types (`services/compact/apiMicrocompact.ts`)

```
type ContextManagementConfig = {
  edits: ContextEditStrategy[]
}
```

6.1.1 `clear_tool_uses_20250919`

```
{
  type: 'clear_tool_uses_20250919',
  trigger: {
    type: 'input_tokens',
    value: 180_000  // Trigger threshold (DEFAULT_MAX_INPUT_TOKENS)
  },
  keep: {
    type: 'tool_uses',
    value: N         // Keep the most recent N tool calls
  },
  clear_tool_inputs: [
    'Bash', 'Read', 'Grep', 'Glob', 'WebSearch', 'WebFetch'
  ],
  clear_at_least: {
    type: 'input_tokens',
    value: 140_000  // Clear at least 140K tokens (180K - 40K)
  },
  exclude_tools: ['Edit', 'Write', 'NotebookEdit']  // Don't clear edit-class tools
}
```

Trigger timing: When `input_tokens >= trigger.value` , the server automatically executes cleanup down to the `keep` limit.

6.1.2 `clear_thinking_20251015`

```
{
  type: 'clear_thinking_20251015',
  keep: 'all' | { type: 'thinking_turns', value: N }
// Keep the most recent N thinking blocks (or all)
// Skip when redact-thinking is active (because edited blocks have no model-visible content)
}
```

6.2 Usage Scenarios

- **Thinking cleanup**: Used by default in all sessions with thinking enabled but redact-thinking not enabled
- **Tool result cleanup**: Enabled via `USE_API_CLEAR_TOOL_RESULTS` / `USE_API_CLEAR_TOOL_USES` environment variables (ANT users only)

PS:

- redact-thinking: "Redacts" (edits) thinking blocks — replaces model-generated thinking content with unreadable `redacted_thinking` blocks. Mainly for privacy/security and token billing concerns
- ANT users: Short for Anthropic employees, can test features still in experimentation

---

7. Full Compaction

When all lightweight mechanisms are insufficient to control context size, Claude Code executes full compaction: generating a structured summary using an AI model to replace the entire conversation history.

7.1 Trigger Conditions

Triggered by token count

```
tokenCountWithEstimation(messages) >= getAutoCompactThreshold(model)
// = effectiveContextWindow - AUTOCOMPACT_BUFFER_TOKENS(13K)
// Approximately 93.5% of effectiveWindow
```

PS: effectiveWindow = Model context window - Model max output tokens

7.2 Complete Execution Flow

```
Input: messages[] (current full conversation history)
Output: CompactionResult

Phase 1: Pre-processing
  ↓
  executePreCompactHooks()  ← Run pre-compaction hooks (allow custom directives)
  ↓
Phase 2: Summary Generation (AI Model Call)
  ↓
  stripImagesFromMessages()   ← Strip images/documents
  stripReinjectedAttachments()← Strip reinjectable attachments (two types:
                                skill_discovery signal (tells model which skills
                                are available) and skill_listing content. Stripped
                                because these attachments will be reinjected after compaction)
  ↓
  streamCompactSummary()       ← Call model to generate summary
      ├── Priority: fork sub-agent path (shared prompt cache)
      └── Fallback: direct streaming call
  ↓
  PTL Retry (up to 3 times):
      truncateHeadForPTLRetry()  ← If the compaction request itself exceeds context
  ↓
Phase 3: State Cleanup
  ↓
  context.readFileState.clear()         ← Clear file read cache (after each Read tool call,
                                          file content is cached. Clearing applies to all files
                                          rather than selective clearing, for: 1. Prevent
```

```
                                          duplication between compaction result and re-read
content;
                                          2. Files may have been modified by user or environment,
                                          force reading the latest version; 3. After compaction,
                                          conversation history becomes a summary, and the model no
                                          longer needs content of files the "model claims to have
read")
  context.loadedNestedMemoryPaths.clear()← Clear nested memory paths (this Set records which nested
                                          CLAUDE.md files have been injected into the conversation
                                          during this session; clearing is for deduplication)
  ↓
Phase 4: Re-inject Attachments (post-compaction info backfill)
  ↓
  Recently accessed state, maintain semantic continuity
  createPostCompactFileAttachments()    ← Restore recently accessed files (max 5, total 50K)
  Restore state
  createAsyncAgentAttachmentsIfNeeded()  ← Restore async agent state
  createPlanAttachmentIfNeeded()         ← Restore plan file
  createPlanModeAttachmentIfNeeded()    ← Restore plan mode reminder
  createSkillAttachmentIfNeeded()       ← Restore invoked skills (5K per skill); how to determine
invoked skills?
  Restore environment info
  getDeferredToolsDeltaAttachment()      ← Re-announce tool changes
  getAgentListingDeltaAttachment()        ← Re-announce agent list
  getMcpInstructionsDeltaAttachment()  ← Re-announce MCP instructions
  ↓
Phase 5: Create Compaction Boundary
  ↓
  createCompactBoundaryMessage()        ← Create system boundary message (update location marker,
                                          mark which content before was compacted)
  ↓
Phase 6: SessionStart Hooks
  ↓
  processSessionStartHooks('compact')  ← Reload CLAUDE.md and other context (doing this separately
                                          after boundary creation is because the process modeling
                                          differs: tools/Agent/MCP deltas are built in real-time
                                          during compaction and put into the messages array;
Memory /
                                          CLAUDE.md are frozen in systemPrompt before
                                          compactConversation() is called. If changed during
                                          compaction, it only takes effect on the next API request
                                          — so we wait for the next turn. It's not for the current
                                          API call, but for the next getSystemPrompt() call —
                                          ensuring the next turn gets fresh memory content)
  ↓
Phase 7: Event Logging and Cleanup
  ↓
  notifyCompaction()                    ← Notify cache-break detection (to prevent cache-break
                                          detection from false positives after compaction: after
                                          compaction, message count legitimately drops
significantly,
                                          and cache_read_tokens will also drop significantly —
this
                                          is normal behavior, not a break)
  markPostCompaction()                  ← Set post-compaction state
  reAppendSessionMetadata()             ← Refresh session metadata
  writeSessionTranscriptSegment()       ← Write session transcript (KAIROS feature)
```

```
  executePostCompactHooks()             ← Run post-compaction hooks
  runPostCompactCleanup()              ← Clean up cache and state
```

7.3 Compaction Summary Prompt Design

Compaction summarization requires the model to generate highly structured output — this is one of the most critical parts of the entire compaction system.

7.3.1 Prompt Template Structure

The prompt contains three parts:

1. Mandatory No-Tools Lead (CRITICAL)

```
CRITICAL: Respond with TEXT ONLY. Do NOT call any tools.
- Do NOT use Read, Bash, Grep, Glob, Edit, Write, or ANY other tool.
- You already have all the context you need in the conversation above.
- Tool calls will be REJECTED and will waste your only turn — you will fail the task.
- Your entire response must be plain text: an <analysis> block followed by a <summary> block.
```

Extremely important: Only reply with plain text. Do not call any tools.

- Do not use Read, Bash, Grep, Glob, Edit, Write, or any other tools.
- You already have all the context you need in the conversation above.
- Tool calls will be rejected and will waste your only turn — this will cause your task to fail.
- Your entire response must be plain text: one block followed by another block.

Design rationale: The compaction fork sub-agent inherits the parent session's full toolset, but the model may still attempt tool calls. This lead explicitly prohibits tool usage, ensuring the model only outputs text.

PS: During previous OpenClaw testing, it was found that the model has some probability of not following compaction instructions and choosing to continue executing follow-up tasks instead.

2. Detailed Analysis Instructions (DETAILED_ANALYSIS_INSTRUCTION)

The model is required to do the following in the `<analysis>` block:

- Analyze each message in chronological order → Overcome the model's "attention loss" and "context forgetting" problems in long conversations, prevent logical discontinuities
- Identify: explicit requests, intentions, code snippets, file names, function signatures → Entity extraction, convert to structured information
- Record: encountered errors, user corrections, methods no longer tried → Negative memory, prevent repeated pitfalls
- Ensure technical accuracy and completeness

3. Summary Format Requirements (9 Required Sections)

PS: OpenClaw's more general agent has similar sections, but **important code snippets** are replaced with **important context snippets**.

```
1. Primary Request and Intent:
   All explicit requests and intents from the user

2. Key Technical Concepts:
   List of important technical concepts, frameworks, patterns

3. Files and Code Sections:
   - File name
     - Why this file is important
     - What modifications were made to it (if any)
```

```
      - Important code snippets
    - ...

 4. Errors and Fixes:
    - Error description:
      - Fix method
      - User feedback on this error (if any)
    - ...

 5. Problem Solving:
    Problems already solved + ongoing investigation

 6. All User Messages:
    All user messages that are not tool results (in chronological order)

 7. Pending Tasks:
    Explicitly requested but not yet completed tasks

 8. Current Work:
    Specific work in progress before compaction
    (Include file names, code snippets)

 9. Optional Next Step:
    Next step that directly continues the most recent work
    (Must reference specific content from the original conversation)
```

7.3.2 Output Parsing

```
// Extract summary from model response in compact.ts:
const summary = getAssistantMessageText(summaryResponse)

// The <analysis> block is stripped during parsing; only the <summary> portion is kept
// This is because <analysis> is a draft process and should not enter the context
```

7.3.3 Summary Result Storage

```
// Compaction summary stored as a user message (role='user')
createUserMessage({
  content: getCompactUserSummaryMessage(summary, suppressQuestions, transcriptPath),
  isCompactSummary: true,              // Marked as compaction summary
  isVisibleInTranscriptOnly: true,   // Only visible in transcript, not sent to API
  summarizeMetadata: {
    messagesSummarized: N,
    userContext: string,
    direction: 'from' | 'up_to',
  }
})
```

Note: `isVisibleInTranscriptOnly` ensures the summary **will not enter the next API request** (normalizeMessagesForAPI filters it out), but will appear in the user-visible transcript.

7.4 Image Stripping Processing

```
stripImagesFromMessages(messages: Message[]): Message[]
```

Images in user messages are replaced before being fed into compaction (placeholder substitution):

- `image` blocks → `[image]` text blocks
- `document` blocks → `[document]` text blocks
- **Images/documents nested in tool results**: Processed the same way

Rationale: Images do not directly help generate conversation summaries. However, images can cause the compaction API call itself to exceed context limits (users in CCD sessions frequently attach images), so text markers are used instead.

7.5 File Recovery After Compaction

```
createPostCompactFileAttachments(
  preCompactReadFileState,  // All file reads tracked before compaction
  context,
  maxFiles: 5,              // POST_COMPACT_MAX_FILES_TO_RESTORE
  preservedMessages?,       // Preserved messages (avoid duplicate injection)
): AttachmentMessage[]
```

File selection strategy:

1. Extract all files from `readFileState` (sorted by most recent access time)
2. Exclude plan files (`getPlanFilePath()`) and all memory files
3. Exclude files that already have Read results in `preservedMessages` (avoid duplication)
4. Take the most recent 5

Token budget control:

```
POST_COMPACT_TOKEN_BUDGET = 50_000   // Max 50K tokens total
POST_COMPACT_MAX_TOKENS_PER_FILE = 5_000  // Max 5K per file
// When a file exceeds 5K, truncate from the beginning with a marker
```

Token savings from file recovery: If not recovered, the model would need to re-read these files after each compaction. At an average of 500 tokens per Read and 5 files, approximately 2.5K tokens are saved per compaction.

7.6 Skill Recovery

```
createSkillAttachmentIfNeeded(agentId?): AttachmentMessage | null

// Skill content is key information that persists across compaction
// Max 5K tokens per skill
// Total skill budget: 25K tokens
// Sorted by invocation time (newest first), oldest discarded when budget exceeded
```

Why not clear sentSkillNames: Re-injecting the complete skill listing (~4K tokens) is pure cache creation with no actual benefit. Invoked skill content is preserved via `invoked_skills` attachment, and new skill dynamic additions are handled by `skillChangeDetector`.

PS: InvokedSkillInfo retains "invocation timestamp" and "which agent invoked it". Invoked skills retain the original version because after compaction, the skill file path remains the same, but the file content may have been modified by the user. Keeping a snapshot at compaction time ensures consistency.

7.7 PTL Retry Mechanism

PS: Using the fixed value of 135K to trigger prompt-too-long judgment.

```
// When the compaction request itself triggers prompt_too_long
truncateHeadForPTLRetry(messages, ptlResponse): Message[] | null
```

```
// Max 3 retries (MAX_PTL_RETRIES = 3)
// Retry strategy:
// 1. Parse tokenGap from API error
// 2. Group by API round, start discarding from the oldest group
// 3. Discard until tokenGap is covered
// 4. Fallback: if tokenGap cannot be parsed, discard 20% of groups directly

// Retry marker: '[earlier conversation truncated for compaction retry]'
// If the first message is this marker, strip it first before grouping (avoid infinite loop)
```

7.8 Prioritizing Fork Sub-Agent Path During Compaction (Subprocess)

```
// compact.ts - streamCompactSummary priority path
if (promptCacheSharingEnabled) {
  const result = await runForkedAgent({
    promptMessages: [summaryRequest],
    cacheSafeParams,        // Inherit parent session's cache params
    canUseTool: createCompactCanUseTool(),  // Disable tools
    querySource: 'compact',
    forkLabel: 'compact',
    maxTurns: 1,            // Only allow 1 turn
    skipCacheWrite: true,   // Don't write new cache (current context is already cached)
    // Pass compaction context's abortController: user Esc can interrupt
  })
  // Extract summary text from result messages
}
```

Why pass parent session's cacheSafeParams: The compaction fork needs to use exactly the same cache key as the main session (system prompt + tools + message prefix) to hit the prompt cache already created by the main session. If the cache key doesn't match, the fork's cache hit rate is extremely low (approximately 98% miss rate for 3P users).

PS: The system prompt during compaction seems to remain unchanged.

Fork path (priority):

Direct streaming path (fallback):

7.9 Direct Streaming Path (Fallback)

When the fork path fails or cache sharing is disabled, direct streaming call is used:

```
// Tool list: FileReadTool only (+ ToolSearchTool if enabled)
// Key point: do not set maxOutputTokens (avoid mismatch with parent session cache key)
// Alternative: set in options.maxOutputTokensOverride
```

PS: During compaction, directly call the API in the current process, receive summary via streaming.

---

8. Session Memory

Session memory is a **preventive** context management mechanism. It continuously maintains a Markdown note in the background during the conversation, extracting key information. This enables compaction to replace the entire conversation history with "session notes + preserved tail messages" while retaining substantial context. → Unload context information to the filesystem.

8.1 Core Difference from Compaction

	Full Compaction	Session Memory
Timing	When context pressure is too high	Continuously maintained during conversation
Method	Generate summary from scratch	**Incrementally update** existing notes
Content	Regenerated each compaction	Gradually accumulated
API call	Required	Required (**sub-agent updates**)
Coverage	Depends on model summary quality	Incremental accumulation, more comprehensive

8.2 Trigger Conditions

```
// sessionMemoryUtils.ts
DEFAULT_SESSION_MEMORY_CONFIG = {
  minimumMessageTokensToInit: 10_000,    // Init: context at least 10K tokens
  minimumTokensBetweenUpdate: 5_000,     // Update: at least 5K token growth since last time
  toolCallsBetweenUpdates: 3,            // Update: at least 3 tool calls
}
```

Trigger conditions (must be **all satisfied**):

```
shouldExtractMemory(messages: Message[]): boolean {
  // 1. Already initialized (minimumMessageTokensToInit met)
  // 2. Token growth >= minimumTokensBetweenUpdate
  // 3. Tool call count >= toolCallsBetweenUpdates
  // 4. Last assistant message has no tool calls (natural conversation pause point)

  const shouldExtract =
    (hasMetTokenThreshold && hasMetToolCallThreshold) ||
    (hasMetTokenThreshold && !hasToolCallsInLastTurn)
}
```

Why require "no tool calls at the end": This ensures extraction happens during natural conversation gaps, not in the middle of tool calls, to avoid missing context.

8.3 Sub-Agent Execution Flow

```
Main conversation loop (after each response)
    ↓
postSamplingHook: extractSessionMemory
    ↓
Check trigger conditions (shouldExtractMemory)
    ↓
setupSessionMemoryFile()
    ├─ Create ~/.claude/sessions/<id>/session-memory.md (if not exists)
    ├─ Initialize with template
    └─ FileReadTool reads current content
    ↓
buildSessionMemoryUpdatePrompt(currentMemory, memoryPath)
    ├─ Load custom prompt.md or use default
    ├─ Analyze token size of each section
    └─ Generate warning if over limit
    ↓
runForkedAgent({
  promptMessages: [userPrompt],
  canUseTool: createMemoryFileCanUseTool(memoryPath),  // Only allow Edit on this file
  querySource: 'session_memory',
```

```
  forkLabel: 'session_memory',
  maxTurns: 1,    // Usually completes in 1 turn (model directly Edits)
})
    ↓
Record token usage
Update lastSummarizedMessageId
```

PS: The sub-agent for session memory reuses the main agent's system prompt, meaning KV cache can be hit. Additionally, Claude supports multiple cache pointer flags.

8.4 Session Memory Template Details

```
# Session Title
_A short, recognizable title of 5-10 words_
(Example: Optimize PostgreSQL query performance)

# Current State
_What is currently being worked on? Uncompleted pending tasks. Next steps._
(This is the most critical section; first thing checked after compaction)

# Task Specification
_What is the user asking to build? Design decisions and other context._

# Files and Functions
_What are the important files? Brief descriptions of their content and relevance._
(Includes file paths, function signatures, key logic)

# Workflow
_Common bash commands and their order? How to interpret output?_
(Deployment flow, test commands, startup commands, etc.)

# Errors & Corrections
_Encountered errors and how they were fixed. What did the user correct? What failed?_
(Includes error messages, fix steps, user feedback)

# Codebase and System Documentation
_Important system components? How they work/compose?_

# Learnings
_What worked well? What didn't? What to avoid?_

# Key Results
_Specific user-requested output (answers, tables, documents). Exact repeat._
(Preserve complete tables, code snippets, answers)

# Worklog
_Brief record of each step._
(Concise timeline for review)
```

PS: From prior experience, the following sections can be added:

```
# Global Constraints & Preferences
_Behavioral boundaries and absolute no-nos_
(Example: Never directly modify main branch, output must strictly follow a specific style)

# Environment Context
_Specific system and external environment context required for current task execution_
```

```
(Includes database connection string format, specific port numbers, system and key dependency
versions)
```

8.5 Token Limits

```
// sessionMemory/prompts.ts
MAX_SECTION_LENGTH = 2000          // Max 2000 tokens per section
MAX_TOTAL_SESSION_MEMORY_TOKENS = 12000  // Max 12K tokens total

// When exceeding limits:
// 1. Oversized sections: generate "Section exceeds limit, please condense" warning
// 2. Total tokens over limit: append "File exceeds 12K tokens, please condense all sections"
// During compaction, use truncateSessionMemoryForCompact():
//    Max 8000 characters per section retained
//    Excess truncated with "[content truncated]" marker
```

PS:

Scenario 1: Sub-agent updating session memory

Let the sub-agent condense itself; the prompt the sub-agent receives will have extra text at the end:

```
  Oversized sections to condense:
  - "# Errors & Corrections" is ~3500 tokens (limit: 2000)
  - "# Key results" is ~2800 tokens (limit: 2000)

  CRITICAL: The session memory file is currently ~14000 tokens,
  which exceeds the maximum of 12000 tokens.
```

Scenario 2: Injecting session memory during compaction

Call truncateSessionMemoryForCompact → directly truncate via function, not through sub-agent

```
  Per-section character limit = MAX_SECTION_LENGTH × 4 = 2000 × 4 = 8000 characters
                      ↑ roughTokenCountEstimation uses length/4
                      so character limit = 2000 tokens × 4 chars/token

  Processing:
    1. Split each section by '# '
    2. Keep each section line by line from the beginning
    3. Keep until character limit is reached
    4. Add truncation marker

  Original content (a section):
    # Errors & Corrections
    _Encountered errors and fixes..._
    - Error1: reason, fix
    - Error2: reason, fix
    - Error3: reason, fix  ← exceeds limit
    - Error4: reason, fix  ← truncated
    - Error5: reason, fix  ← truncated

  After truncation:
    # Errors & Corrections
    _Encountered errors and fixes..._
    - Error1: reason, fix
    - Error2: reason, fix
    - Error3: reason, fix
```

```
    [... content truncated; use Read on the skill path if you need the full text]
    ↑ Truncation marker, telling the model it can re-read the skill file
```

8.6 Custom Template Support

```
// ~/.claude/session-memory/config/template.md  → Session memory template
// ~/.claude/session-memory/config/prompt.md    → Update prompt
// Custom prompt uses {{currentNotes}} and {{notesPath}} variables
```

8.7 Tool Usage Restrictions

```
createMemoryFileCanUseTool(memoryPath): CanUseToolFn
// Only allowed: Edit tool, and only if file_path === memoryPath
// Denied: all other tools
// This is an extremely restricted environment, preventing sub-agents from making unexpected
operations
```

8.8 Mutex with Main Agent

Mutex rules between session memory sub-agent and main agent:

- **Main agent writes**: Skip sub-agent's extraction this round, advance the cursor
- **Sub-agent writes**: Main agent will not write again
- **Both attempt to write simultaneously**: Cannot happen, as they run in different processes

8.9 Usage of Session Memory

The main agent does not proactively read this file. It is **only used during compaction**.

```
autoCompact / manual compact
    └── trySessionMemoryCompaction()
          ├── getSessionMemoryContent()         ← Read file directly
          ├── isSessionMemoryEmpty()           ← Check for real content
          ├── calculateMessagesToKeepIndex()   ← Calculate preservation range based on
lastSummarizedMessageId
          └── createCompactionResultFromSessionMemory()
                ├── truncateSessionMemoryForCompact()   ← Truncate oversized content
                ├── getCompactUserSummaryMessage()       ← Wrap content into summary user message
                └── buildPostCompactMessages()          ← Add to message history

  After compaction, session memory file content enters the main agent's messages list
  through the summary user message, indirectly becoming part of the conversation context
```

---

9. Session Memory Compaction (SM-Compact, Important)

Session Memory Compaction (SM-Compact) is an **alternative** to full compaction, using incrementally maintained session notes instead of summaries generated from scratch each time.

9.1 Comparison with Traditional Compaction

Dimension	Traditional Compaction	SM-Compaction
Summary source	Generated from scratch each time	Incrementally maintained session notes
History processing	All replaced with summary	Selectively preserve tail
API call	Required (model generates summary)	**Not required**

Dimension	Traditional Compaction	SM-Compaction
Preserved content	Summary text only	Summary + most recent 10K-40K tokens
Applicable scenario	General	Long multi-round sessions

PS:
SM-Compaction has higher priority than traditional compaction. When SM-Compact fails, it falls back to traditional compaction.

SM-Compaction advantages:

```
  Traditional compactConversation requires each time:
  fork agent → call summarization API → wait for response → generate summary

  SM-Compaction only needs:
  directly read summary.md file → generate summary user message → return

  No API call, zero latency, zero cost
```

9.2 Preserved Message Calculation

```
// services/compact/sessionMemoryCompact.ts
calculateMessagesToKeepIndex(messages, lastSummarizedIndex): number

// Starting from lastSummarizedMessageId, expand backward:
// 1. Meet minTokens (default 10K)
// 2. Meet minTextBlockMessages (default 5)
// 3. Don't exceed maxTokens (default 40K)
// 4. floor = last compaction boundary + 1 (don't cut inside compaction boundary)
```

`adjustIndexToPreserveAPIInvariants` protection:

The API has two hard constraints that compaction must satisfy:

Constraint 1: Don't split tool_use/tool_result pairs

Constraint 2: Don't split streaming messages sharing message.id
PS: This is caused by API behavior in streaming mode. Specifically, in streaming mode, an assistant response arrives in segments. Therefore, if any block with a given message.id is preserved, all blocks with that message.id must be preserved — because the API ultimately merges by message.id.

```
Claude's streaming emits separate assistant messages for each content block:
  Index N:   assistant(id=X), content: [thinking]      ← streaming chunk 1
  Index N+1: assistant(id=X), content: [tool_use: ID]  ← streaming chunk 2
  Index N+2: user, content: [tool_result: ID]

If startIndex = N+1 (only keep tool_use but discard thinking):
  → when normalizeMessagesForAPI merges by message.id
  → thinking block is lost
  → model loses important reasoning process

Correct approach:
  → detect that N+1 shares message.id with N
  → adjust startIndex to N (include complete streaming group)
```

9.3 Feature Availability Check

Trigger condition: `effective context window × 90%` (traditional compaction trigger threshold is `effective context window × 93.5%`)

Multi-layer progressive check for whether `Session Memory Compact` feature can be used:

```
shouldUseSessionMemoryCompaction(): boolean
// 1. `tengu_session_memory` feature flag = true; main switch: whether session memory itself is enabled
// 2. `tengu_sm_compact` feature flag = true; sub switch: whether session memory-driven compaction is enabled
// 3. Check environment variables (feature switches): ENABLE_CLAUDE_CODE_SM_COMPACT (force enable)
//    or DISABLE_CLAUDE_CODE_SM_COMPACT (force disable)
// 4. Session memory file exists and is non-empty (isSessionMemoryEmpty())
```

9.4 Execution Flow

```
trySessionMemoryCompaction(messages, agentId, autoCompactThreshold)
    ↓
1. Wait for any in-progress session memory extraction (15s timeout)
    ↓
2. Get session memory file content
    ↓
3. Locate lastSummarizedMessageId (try to recover if missing)
    ↓
4. Calculate message range to preserve (calculateMessagesToKeepIndex)
    ↓
5. Filter out old compaction boundary messages
    ↓
6. Execute SessionStart Hooks (reload CLAUDE.md)
    ↓
7. Create compaction result
    ├─ createCompactBoundaryMessage()
    ├─ Convert session memory content to summary message
    ├─ truncateSessionMemoryForCompact() (ensure within budget)
    └─ Attach plans, etc.
    ↓
8. Check postCompactTokenCount < autoCompactThreshold
    ↓ Pass → return result
    ↓ Not pass → return null, fall back to traditional compaction
```

9.5 Compaction Boundary Metadata

```
// compact.ts
annotateBoundaryWithPreservedSegment(
  boundary,
  anchorUuid,   // Anchor: last summary message
  messagesToKeep,
): SystemCompactBoundaryMessage

// preservedSegment tells session storage how to restore chain structure:
// Points to original messages in physical storage, used to tell session storage
// how to rebuild the association before and after compaction

// headUuid → first message in preserved segment; UUID of the oldest replaced message
// anchorUuid → summary message (mount point); UUID of new summary message generated by summarizer
// tailUuid → last message in preserved segment; UUID of last message in preserved segment
```

10. Auto-Compact

Auto-compact is a compaction mechanism that automatically triggers when context approaches the threshold, without requiring the user to manually execute `/compact`.

10.1 Threshold System Explained

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Effective Context Window                      │
│                                                                      |
│  0%        warning          autocompact        blocking     100%    │
│   ├──────────┤├──────────────┤├─────────────────┤├────────────┤      │
│   0         180K            187K              197K        200K      |
│             ↑               ↑                ↑                      |
│      isAboveWarning    shouldAutoCompact  isAtBlocking              |
│       Threshold           → Auto-Compact    Limit → Block Input     |
│                                                                      |
│  WARNING_BUFFER = 20K    AUTO_BUFFER = 13K  MANUAL_BUFFER = 3K      |
└──────────────────────────────────────────────────────────────────────┘
```

10.2 Warning States

```
calculateTokenWarningState(tokenUsage, model): {
  percentLeft: number,                    // Remaining space percentage
  isAboveWarningThreshold: boolean,        // Issue warning (yellow)
  isAboveErrorThreshold: boolean,          // Issue error (red)
  isAboveAutoCompactThreshold: boolean,   // Trigger auto-compact
  isAtBlockingLimit: boolean,             // Block input
}
```

10.3 Compaction Priority

```
autoCompactIfNeeded(messages, toolUseContext, cacheSafeParams, ...): {
  wasCompacted: boolean,
  compactionResult?: CompactionResult,
  consecutiveFailures?: number,
}

// Execution order:
// 1. Check circuit breaker (consecutive failures >= 3 → return)
// 2. shouldAutoCompact() → check threshold
// 3. Try SM-Compact (trySessionMemoryCompaction)
 //    → Success → return
// 4. Execute traditional compaction (compactConversation)
 //    → Success → reset circuit breaker consecutiveFailures = 0
 //    → Failure → consecutiveFailures++
```

10.4 Circuit Breaker Explained

Circuit breaks after three consecutive failures.

```
const MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3

// Data background (2026-03-10):
// 1,279 sessions had 50+ consecutive failures (max 3,272)
```

```
// Approximately 250K wasted API calls per day

// On consecutive failures:
// - Skip futile compaction attempts
// - User receives "conversation full" error, needs manual intervention (/clear or /compact with options)
// - Next new input re-checks threshold
```

10.5 Disable Control Matrix

Control Method	Effect	Priority
`DISABLE_COMPACT=1`	Disable all compaction	Strongest
`DISABLE_AUTO_COMPACT=1`	Only disable auto-compact, retain `/compact`	Strong
`userConfig.autoCompactEnabled=false`	User configuration	Medium
`CONTEXT_COLLAPSE` feature	Collapse system takes over, disables auto-compact	Medium
`REACTIVE_COMPACT` feature	Disable proactive compaction, rely on 413 response	Medium

10.6 Special Case of Marble Origami

When `CONTEXT_COLLAPSE` is on, `marble_origami` (ctx-agent)'s auto-compact is disabled:

- If ctx-agent context blows up, auto-compact triggers
- `runPostCompactCleanup` calls `resetContextCollapse()`
- This would corrupt **main thread**'s committed log (shared module state)

Solution: When `CONTEXT_COLLAPSE` is on, ctx-agent disables auto-compact.

PS:

- marble_origami is a ctx-agent (context collapse agent) running in a sub-agent/fork. It shares module-level state in the same process as the main thread
- marble_origami is a shared-process-state ctx-agent. If its auto-compact succeeds, it triggers `resetContextCollapse()` — even with the isMainThreadCompact check, the existence of shared state itself is unsafe. So marble_origami is excluded at the source of shouldAutoCompact. The ctx-agent's context explosion problem is handled by the Context Collapse system itself, not the auto-compact path.

---

11. Context Collapse

Context Collapse (`CONTEXT_COLLAPSE` feature flag) is an **experimental** context management mechanism controlled via feature gates.

PS:

- Context Collapse is a completed implementation currently in internal gradual rollout testing; it has not been officially released. There is no public documentation or stable product naming
- Since the core service source code (services/contextCollapse/) is not included in restored-src/, only complete logic inference through cross-analysis of 14 reference points

11.1 Core Difference from Compaction

PS: Collapse is implemented in code, without calling AI.

```
Compaction:
  Discard old messages → replace with summary
  Information loss: Medium (controlled by prompt quality)
  Latency: High (requires AI model call for summary generation)
```

```
  Precision: Low-Medium (summary quality is inconsistent)

Context Collapse:
  Selective discarding → precise control over preserved content
  Information loss: Precisely controllable
  Latency: Low (no AI call needed)
  Precision: High (precise proportional discarding)
```

11.2 Design Principles

Similar to OS memory management paging strategy:

- **90% threshold**: Start commit operation — write intermediate results to disk
- **95% threshold**: Block new input — wait for commit to complete
- **Preservation strategy**: Keep the most recent N% of messages, ensuring working context is not lost

11.3 Mutual Exclusion with Auto-Compact

Priority ordering: **Context Collapse > Session Memory Compact > Auto-Compact**

```
When CONTEXT_COLLAPSE is enabled:
  - Auto-Compact is fully suppressed (avoid race conditions)
  - 90% commit threshold < 93.5% auto-compact threshold
  - Collapse system takes over before auto-compact
  - 95% blocking threshold > 93.5% auto-compact threshold

When CONTEXT_COLLAPSE is disabled:
  - Auto-Compact operates normally
  - Relies on summarization for context management
```

11.4 Implementation Location

```
// Dynamic import (avoid circular dependency):
if (feature('CONTEXT_COLLAPSE')) {
  const {
    isContextCollapseEnabled,
    applyCollapsesIfNeeded,
    recoverFromOverflow,
    isWithheldPromptTooLong,
    resetContextCollapse,
  } = require('../services/contextCollapse/index.js')
}
```

Why dynamic import: This module is located in the `services/contextCollapse/` directory, and the file list is not included in the restored-src/ in source map recovery. This indicates that `contextCollapse` is a compiled independent module loaded lazily via `require()`.

11.5 Objects Being Collapsed

Operations are performed on **local context** for each message span (span), so the concept is still collapse. Specifically:

```
  user message → assistant message → user message → ... → assistant message
   ↑                                                          ↑
   firstArchivedUuid (span start)                          lastArchivedUuid (span end)
```

Content NOT collapsed (kept as-is):

- System prompt / appendSystemPrompt
- Tool result messages (raw tool output from API) -> This is not collapsed?? So what exactly is collapsed
- Cache metadata after microcompact
- Most recent N "protected tail" messages — defined by getMessagesAfterCompactBoundary

Collapse product: Each collapsed span is replaced with a metadata placeholder:

```
<collapsed id="16-char-collapseId">Summary text</collapsed>
```

11.6 Two-Phase Collapse Algorithm

Context Collapse is **preventive and progressive**.

Phase 1 — Staging (Pre-collapse)

Trigger condition: 90% effective context window
effective window tokens ≥ effective window × 90%

Triggered at this point:

1. ctx-agent spawn (sub-process, codename "marble_origami") is started
2. ctx-agent analyzes all message spans from "protected tail boundary" to current position
3. **Generates summary for each span**: { startUuid, endUuid, summary, risk, stagedAt }
4. Results written to staged queue (in memory)
5. A marble-origami-snapshot entry is written to transcript (persist staged queue state)
6. Staged spans are not immediately deleted from API context, just waiting in the queue

> Significance of staging: Prepare summaries in advance, don't wait until overflow to start processing.

Phase 2 — Commit (Formal collapse)

Two trigger condition paths:

Path A (Proactive) — 95% blocking trigger
effective window tokens ≥ effective window × 95%
Before sending the next spawn request, check if already staged:

- If staged → directly commit (use existing summaries, no model re-call needed)
- If not staged → immediately generate summaries then commit

Path B (Reactive) — 413 overflow recovery
API returns 413 (prompt too long)
Call `recoverFromOverflow()`:

1. Commit all spans in staged queue (drain)
2. Retry API request with new post-commit context
3. If retry again gets 413 → fall back to reactiveCompact

11.7 Relationship with Other Mechanisms

---

12. Context Analysis

12.1 Use Cases

`analyzeContext()` scans the message array and generates detailed statistics on token consumption, used for:

1. **Telemetry reporting**: Report detailed dimensions of `tengu_compact` event via Statsig

2. **Diagnostic analysis**: Identify optimizable waste points (e.g., duplicate file reads)

PS:

- analyzeContextUsage() is not just for token consumption statistics; it is a complete context audit system — responsible for sensing, classifying, quantifying, and driving actionable optimization suggestions
- `tengu` is Anthropic's internal project codename prefix, spanning the entire codebase (1051 references, 250 files). In Japanese folklore, Tengu is a highly capable spirit. This codename suggests the importance of this project to Anthropic internally.

12.2 Statistical Dimensions

```
type TokenStats = {
  toolRequests: Map<string, number>      // Token consumption for each tool's requests
  toolResults: Map<string, number>       // Token consumption for each tool's results
  humanMessages: number                   // Total user message tokens
  assistantMessages: number              // Total assistant message tokens
  localCommandOutputs: number            // Bash command output tokens
  other: number                          // Other (thinking, attachments, etc.)
  attachments: Map<string, number>         // Count of each attachment type
  duplicateFileReads: Map<string, {       // Duplicate file reads
    count: number
    tokens: number
  }>
  total: number                          // Total tokens
}
```

12.3 Duplicate Read Detection Algorithm

```
// For multiple reads of the same file, count only 1 time
// Duplicate tokens = avgTokensPerRead × (count - 1)

fileReadStats.forEach((data, path) => {
  if (data.count > 1) {
    const avgTokensPerRead = Math.floor(data.totalTokens / data.count)
    const duplicateTokens = avgTokensPerRead * (data.count - 1)
    stats.duplicateFileReads.set(path, { count, tokens: duplicateTokens })
  }
})
```

Practical application: If a file is read 10 times at 1K tokens each, then `duplicate_read_tokens` = 9K. This means after compaction, the model doesn't need to re-read the file, saving substantial tokens.

12.4 Statsig Metric Mapping

```
tokenStatsToStatsigMetrics(stats): Record<string, number>

// Generates 40+ metrics, including:
// - Token consumption and percentage for each tool
// - Duplicate read tokens and file count
// - Token distribution by message type
// - Attachment type counts
```

Categories overview

```
  ContextData.categories = [
    { name: 'System prompt',        tokens: N },   // system prompt section details
    { name: 'System tools',        tokens: N },   // Built-in tool schemas (ANT users can see per-
tool details)
    { name: 'MCP tools',           tokens: N },   // MCP server tools (after dedup)
    { name: 'MCP tools (deferred)', tokens: N, isDeferred: true },  // Loaded but not token-
consuming
    { name: 'System tools (deferred)', tokens: N, isDeferred: true },
    { name: 'Custom agents',       tokens: N },
    { name: 'Memory files',        tokens: N },   // CLAUDE.md per-file details
    { name: 'Skills',              tokens: N },   // Skill frontmatter (not full content)
    { name: 'Messages',            tokens: N },   // Conversation history (after microcompact)
    { name: 'Autocompact buffer',   tokens: N },   // Safety space reserved for autocompact
(skipped when CC is on)
    { name: 'Free space',          tokens: N },
  ]
```

Key: Categories with `isDeferred: true` are not counted in actualUsage — they represent parts that are "loaded but only consume tokens on demand."

Deferred Tool:

- Only tells the model the tool name, not the full schema; loaded on demand
- Important note: User-configured MCP tools are the primary target for deferred tools: MCP tools are deferred by default (isMcp === true takes that branch), while only explicitly marked shouldDefer built-in tools are deferred (some built-in tools prohibit deferral)
- Normally, every API request includes the full schema (name + description + parameters) for all tools. The Deferred mechanism changes this:

```
  Normal tools (non-deferred)
    → API request includes full schema (token consumption is immediate)

  Deferred tools
    → API request has only tool name (very few tokens)
    → When model wants to call → first call ToolSearchTool to fetch full schema → then execute
```

The philosophy is similar to progressive disclosure of skills; the code even directly states:

```
  // loadSkillsDir.ts:97
  /**
   * Estimates token count for a skill based on frontmatter only
   * (name, description, whenToUse) since full content is
   * only loaded on invocation.
   */
  export function estimateSkillFrontmatterTokens(skill: Command): number
```

12.5 Detecting Optimizable Waste Points

Waste detection is divided into two systems:
System A: generateContextSuggestions() — User-facing suggestions

System B: checkContextWarnings() — Warnings for doctor command

12.6 Summary

UI layer users see
/context command ← ContextData renders
ContextVisualization ← gridRows + categories renders
TokenWarning ← percentage triggers warning

Suggestion layer
contextSuggestions ← Proactive suggestions (what can be optimized)
doctorContextWarnings ← Warning-type (config/scale issues)

Statistics layer
analyzeContextUsage ← Bucketed counting, API/client dual precision:
Uses server numbers when actual usage from API response is available;
falls back to client estimation otherwise
← Deferred mechanism (saves tokens)
← Per-tool granular breakdown; statistics on "which tool costs the most";
displays message distribution by tool type
← Statistics after microcompact, ensuring statistics reflect
the actual tokens the model sees

---

13. Post-Compaction Cleanup

`runPostCompactCleanup()` executes after every compaction (auto/manual), cleaning up state polluted by the compaction operation.

13.1 Cleanup Contents Explained

```
runPostCompactCleanup(querySource?: QuerySource): void

// Cleanup executed only on main thread compaction:
resetMicrocompactState()              // Reset microcompact state (tool registration, cache edit
queue)
getUserContext.cache.clear()          // Clear user context cache (CLAUDE.md content)
resetGetMemoryFilesCache()            // Clear memory file discovery cache
clearSystemPromptSections()           // Clear system prompt section cache
clearClassifierApprovals()           // Clear classifier approvals (avoid approvals being
forgotten)
clearSpeculativeChecks()              // Clear speculative check state
clearBetaTracingState()               // Clear beta tracing state
clearSessionMessagesCache()           // Clear session message cache
sweepFileContentCache()               // Sweep post-hoc attribution file cache (COMMIT_ATTRIBUTION)

// Context Collapse (main thread only):
if (feature('CONTEXT_COLLAPSE') && isMainThreadCompact) {
  resetContextCollapse()              // Reset context collapse state
}
```

13.2 Key Items NOT Cleared

```
// The following are intentionally NOT cleared (persist across compaction):
// 1. invoked_skills → needed by createSkillAttachmentIfNeeded
// 2. sentSkillNames → re-injecting complete listing is pure waste
```

PS:
invoked_skills records already-executed skills; the skill currently being executed is not yet in invoked_skills. So "partial

execution" is not an issue.
Then what good does it do for the model to know which skills have been used?
LLM's explanation: Preserving the actual content of skills that have already proven valuable. But this explanation seems to have a loophole... already-executed skills may not need to be executed again in the future.

13.3 Main Thread vs. Sub-Agent Differentiation

```
// Sub-agents (agent:*, sdk) also call functions in compact.ts
// But their compaction should not clear main thread's cache
const isMainThreadCompact =
  querySource === undefined ||
  querySource.startsWith('repl_main_thread') ||
  querySource === 'sdk'
```

Sub-agent post-compaction cleanup:

Summary:
After sub-agent compaction:
✅ resetMicrocompactState() — clears its own (isolated)
✅ contentReplacementState reset — clears its own (isolated)
❌ runPostCompactCleanup() — not called (prevents accidental clearing of shared state)

After main thread compaction:
✅ runPostCompactCleanup() — all cleanup items execute

Core principle: Whoever compacts, clears their own isolated state; whoever calls runPostCompactCleanup, clears shared state. Sub-agent compaction is isolated and only responsible for itself.

---

14. Context Loading and Caching

14.1 System Context (`context.ts`)

PS: These are not the system prompt itself, but extra information injected into the system prompt.

Git Status

```
getGitStatus(): Promise<string | null>
// Includes:
// - Current branch
// - Main branch
// - Git user name
// - git status --short (max 2000 characters, truncated if exceeded)
// - Most recent 5 commits

// Cached: Valid for entire session (memoized)
// Skip conditions:
// 1. NODE_ENV=test (avoid test loops)
// 2. Non-git directory (getIsGit() === false)
// 3. CLAUDE_CODE_REMOTE env var (CCR mode, skip to save overhead)
// 4. Git instructions disabled (shouldIncludeGitInstructions())
```

User Context

```
getUserContext(): Promise<{ claudeMd, currentDate }>
// claudeMd: Content loaded from all CLAUDE.md and memory files
// currentDate: Current ISO date
```

```
// Cached: Valid for entire session (memoized)
// Disable conditions:
// 1. CLAUDE_CODE_DISABLE_CLAUDE_MDS=1 (hard disable)
// 2. --bare mode with no explicit --add-dir (skip auto-discovery)
```

Cache Invalidation

```
// When reloading context is needed (e.g., user modified CLAUDE.md):
setSystemPromptInjection(value)  // Set cache breaker
// → getUserContext.cache.clear()
// → getSystemContext.cache.clear()
// → Reload on next call
```

PS: CLAUDE.md is loaded at three moments:

- Moment 1: Session startup
- Moment 2: After compaction (auto-reload)
- Moment 3: User modified CLAUDE.md (incremental update)

14.2 Session Storage (`utils/sessionStorage.ts`)

Message history is stored on disk in JSONL format, supporting chain-based recovery across compaction boundaries:

```
~/.claude/sessions/<session-id>/
├── messages.jsonl           ← Main message history
├── metadata.jsonl          ← Metadata (title, tags)
└── ...
```

Chain recovery: Each message stores a parentUuid pointing to its parent message when saved. During recovery, you traverse from the newest message backward following parentUuid, forming a chain.

---

15. Persistent Memory Extraction

`extractMemories.ts` runs at the end of each complete query loop, writing key information to persistent storage.

15.1 Difference from Session Memory

	Session Memory	**Persistent Memory**
Location	`~/.claude/sessions/<id>/session-memory.md`	`~/.claude/projects/<path>/memory/`
Lifecycle	Current session	**Cross-session**
Content	Key work records of current session	Cross-session useful knowledge
Trigger	Context growth reaches threshold	End of each conversation round (automatic)
Tool restrictions	Only Edit session memory file	Edit/Write in memory directory

"Each complete query loop" means: every time the main agent finishes a conversation round (model gives final response, no tool calls), stop hooks trigger extractMemories.

15.2 Flow

```
extractMemories executes
    ↓
1. Check if main agent has already written to memory file
   → Written → Skip, advance cursor
   → Not written → Continue
```

```
    ↓
2. Throttle check
   → Should not execute this round → Return
   → Should execute → Continue
    ↓
3. Scan existing files in memory directory
    ↓
4. Fork sub-agent
   → Read memory directory
   → Based on conversation delta (new messages after lastMemoryMessageUuid)
   → Generate update prompt
   → Call Edit/Write tools to write new memory files
    ↓
5. Update cursor lastMemoryMessageUuid
```

15.3 Storage Structure and Content

```
~/.claude/projects/<path>/memory/
├── MEMORY.md              ← Index file (links to all topics)
├── auth-implementation.md ← Topic file 1
├── database-schema.md     ← Topic file 2
└── ...
```

Memory Types: Four Types

1. user — User Profile

```
  <name>user</name>
  - Content saved: User's role, goals, responsibilities, knowledge background
  - When to save: When learning user's role preferences, responsibilities, or knowledge
  - How to use: Adjust style and depth when answering questions based on user profile
  - Example:
    user: "I'm a data scientist investigating what logging we have in place"
    → Save: user is a data scientist, currently focused on observability/logging
```

2. feedback — Feedback Preferences

```
  <name>feedback</name>
  - Content saved: Work approach guidance given by user (what to avoid, what to keep doing)
  - When to save: When user corrects ("no not that") or confirms ("yes exactly")
  - Structure: Rule itself → Why → How to apply
  - Example:
    user: "don't mock the database in these tests"
    → Save: integration tests must hit a real database, not mocks.
        Reason: prior incident where mock/prod divergence masked a broken migration
```

3. project — Project State

```
  <name>project</name>
  - Content saved: Ongoing projects, goals, bugs, incidents
  - When to save: When learning who is doing what, why, or by when
  - Structure: Fact → Why → How to apply
  - Note: Convert relative dates to absolute dates ("Thursday" → "2026-03-05")
  - Example:
    user: "we're freezing non-critical merges after Thursday — mobile team is cutting a release
```

```
branch"
    → Save: merge freeze begins 2026-03-05 for mobile release cut.
        Flag any non-critical PR work scheduled after that date
```

4. reference — External System References

```
  <name>reference</name>
  - Content saved: Location of information in external systems
  - When to save: When learning about external resources and their purposes
  - Example:
    user: "check the Linear project 'INGEST' if you want context on these tickets"
    → Save: pipeline bugs are tracked in Linear project 'INGEST'
```

Explicitly NOT saved content

```
  - Code patterns, conventions, architecture, file paths → derivable from reading code
  - Git history, who-changed-what → git blame / git log are authoritative sources
  - Debugging solutions → fix is in the code, commit message has context
  - Already written in CLAUDE.md
  - Ephemeral task details: in-progress work, temporary state
```

Meaning of "ephemeral task" in the memory system:

- In-progress work
- Temporary state
- Current conversation context

Write Format

Each memory write is an independent file with frontmatter:

```
  ---
  name: Memory name
  description: One-line description
  type: user | feedback | project | reference
  ---

  Memory body content...

  MEMORY.md index records pointers:
  - [Memory title](filename.md) — one-line hook
  - [Memory title2](filename2.md) — one-line hook
```

15.4 Sub-Agent Execution

```
// Tool restrictions: createAutoMemCanUseTool(memoryDir)
allow: Read, Grep, Glob, read-only Bash
allow: Edit, Write within memoryDir
deny: All other operations

// Throttle control:
// Execute once every N qualifying turns
// getFeatureValue('tengu_bramble_lintel', 1)
// Default: execute every turn
```

```
// Max turns: 5 (prevent validation rabbit holes)
// skipTranscript: true (don't write to transcript, avoid races)
```

PS:
The extractMemories sub-agent does not have a dedicated system prompt — it uses the parent agent's system prompt (coding assistant), with task instructions passed via user message:

```
You are now running as a memory extraction sub-agent.
Analyze the most recent ~{N} messages above and use them to update your persistent memory system.

Available tools: Read, Grep, Glob, read-only Bash, and Edit/Write tools restricted to paths within
the memory directory.

You must only use content from the most recent ~{N} messages. Do not waste turns on further
investigation or verification.

## Memory Types
[Detailed definitions of user / feedback / project / reference...]

## Content NOT to Save in Memory
[Code patterns / git history / content already in CLAUDE.md...]

## How to Save Memories
1. First read existing memory files to avoid duplicates
2. Write independent files in frontmatter format
3. Update MEMORY.md index (each entry max 150 characters)
```

15.5 Mutex Protection

```
// If the main agent wrote to memory file this round:
hasMemoryWritesSince(messages, lastMemoryMessageUuid)
// → Skip extraction, advance cursor
// → Both are mutex, avoiding conflicts
```

15.6 Exit Wait

```
// Before CLI is exited by user, drainPendingExtraction()
// Wait up to 60 seconds for extraction to complete
// Prevent fork process from being killed during shutdown
```

---

16. History Management

`history.ts` manages the REPL's prompt history (used for ↑ arrow recall and Ctrl+R search).

REPL: Read-Eval-Print Loop, i.e., the interactive terminal interface of Claude Code CLI.

16.1 Key Constants

```
MAX_HISTORY_ITEMS = 100          // Max 100 entries
MAX_PASTED_CONTENT_LENGTH = 1024 // Pinned content length limit
```

16.2 Main Functions

```
addToHistory(entry)                        // Add to history
getHistory()                               // Get current project history
```

```
getTimestampedHistory()                    // Timestamped version (for picker)
removeLastFromHistory()                    // Undo last addition
expandPastedTextRefs()                     // Resolve pasted references
```

---

17. Message Normalization and Boundary Management

17.1 Message Normalization (`normalizeMessagesForAPI`)

This is the core function that converts internal message format to API request format:

```
normalizeMessagesForAPI(messages, tools): (UserMessage | AssistantMessage)[]
```

PS: Converts session structured content into LLM-used context.

Execution steps:

1. **Reorder attachments**: `reorderAttachmentsForAPI()` bubbles attachments to appropriate positions
2. **Filter virtual messages**:
   ```
   // The following types are filtered out, not entering API requests:
   - progress messages
   - system messages (except local_command)
   - synthetic API error messages
   ```
3. **Handle local commands**: `local_command` system messages are converted to user messages (merged into previous user message)
4. **Handle user messages**:
   - Merge consecutive user messages (Bedrock doesn't support consecutive user messages)
   - Strip tool_reference blocks (when tool search is disabled)
   - Strip blocks corresponding to failed attachments (PDF too long/password-protected/invalid)
   - Handle text completions in tool_result (avoid stop sequence issues)
5. **Validate message order**: Ensure the first message has role='user'

17.2 Compaction Boundary Messages

```
// Create compaction boundary
createCompactBoundaryMessage(
  trigger: 'auto' | 'manual',
  preTokens: number,
  lastPreCompactMessageUuid?,
  userContext?,
  messagesSummarized?,
): SystemCompactBoundaryMessage

// Result structure:
{
  type: 'system',
  subtype: 'compact_boundary',
  content: 'Conversation compacted',
  compactMetadata: {
    trigger: 'auto' | 'manual',
    preTokens: number,
    userContext?: string,
    messagesSummarized?: number,
    preservedSegment?: {
      headUuid: UUID,
```

```
      anchorUuid: UUID,
      tailUuid: UUID,
    },
    preCompactDiscoveredTools?: string[],
  },
  logicalParentUuid?: UUID,  // Points to the last message before compaction
}
```

Key fields:

- `logicalParentUuid` : Establishes the relationship between the compaction boundary and the last pre-compaction message, used for session storage's chain recovery
- `preservedSegment` : Tells the loader how to restore the preserved segment when deduplication skipping
- `preCompactDiscoveredTools` : Deferred tool schemas loaded before compaction

17.3 Microcompact Boundary Messages

```
createMicrocompactBoundaryMessage(
  trigger: 'auto',
  preTokens: number,
  tokensSaved: number,
  compactedToolIds: string[],
  clearedAttachmentUUIDs: string[],
): SystemMicrocompactBoundaryMessage
```

17.4 Finding Compaction Boundaries

```
// Find the index of the last compaction boundary message
findLastCompactBoundaryIndex(messages): number

// Get all messages after the compaction boundary
getMessagesAfterCompactBoundary(messages, options?): T[]
// Includes the boundary message itself
// Supports HISTORY_SNIP: snip view also uses this slice
```

---

18. Key Design Decisions

18.1 Why Fork Sub-Agent for Summary Execution?

All background tasks (summary generation, memory extraction) are executed via `runForkedAgent` , core reasons:

1. **Prompt cache reuse**: Fork inherits the main session's full cache prefix (system prompt + tools + context messages), avoiding redundant computation
2. **State isolation**: Sub-agent cannot modify main agent's `readFileState` , skill list, etc.
3. **Interruptible**: User Esc interrupts sub-agent via shared `AbortController` signal
4. **No side effects**: Sub-agent messages don't enter main transcript (`skipTranscript: true`)

18.2 Why Microcompact at API Layer Instead of Direct Message Modification?

Cached Microcompact chooses to operate via `cache_edits` at the API layer rather than directly modifying message content:

1. **Preserve prompt cache**: Directly modifying message content would break the server-side cache, causing recalculation every time
2. **Reduce network load**: API only needs to transmit incremental changes (what to delete), not the entire prefix
3. **Atomicity**: Server-side cache operations are atomic, avoiding race conditions

4. **Complementary to compaction**: Microcompact cleans old results, full compaction handles accumulated pressure

18.3 Why Use GrowthBook for Dynamic Configuration?

All thresholds and switches are dynamically configured via GrowthBook, not hardcoded:

```
getFeatureValue_CACHED_MAY_BE_STALE('tengu_session_memory', false)
getDynamicConfig_CACHED_MAY_BE_STALE('tengu_sm_compact_config', {})
```

- **Zero-delay deployment**: Adjust thresholds without releasing new versions
- **A/B testing**: Different user groups use different configurations
- **Gradual rollout**: Expand feature coverage progressively
- **Non-blocking cache**: `CACHED_MAY_BE_STALE` returns cached value immediately, doesn't block initialization

PS: These parameter thresholds and switches are directly exposed interfaces, adjustable without modifying underlying code.

---

19. Global Data Flow Overview

19.1 Complete Request Processing Flow

Where does session memory compact fit in??

```
User input
    │
    ▼
┌──────────────────────────────────────────────────────────────────┐
│ 1. Microcompact                                                  │
│    microcompactMessages(messages, toolUseContext, querySource)   │
│                                                                   │
│    ┌──────────────────┐    ┌──────────────────────────────┐     │
│    │ Time-based MC    │    │ Cached MC (CACHED_MICROCOMPACT)  │  │
│    │ if (gap >= 60min) │    │ if (cache_edits API available)  │  │
│    │ Clean old tool results │ Delete old tools, keep cache prefix │  │
│    └──────────────────┘    └──────────────────────────────┘     │
│                                                                   │
│    └─ Returns { messages (cleaned), compactionInfo: { pendingCacheEdits } } │
└──────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────┐
│ 2. Snip History Pruning (HISTORY_SNIP feature) -> This is actually off │
│    snipCompactIfNeeded(messages)                                 │
│    ├─ Calculate snip point (by token or message count)           │
│    ├─ Replace with SNIP_MARKER                                   │
│    └─ Return tokensFreed (passed to autocompact)                 │
└──────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────┐
│ 3. Token Counting & Threshold Detection                          │
│    tokenCountWithEstimation(messages) - snipTokensFreed          │
│    vs getAutoCompactThreshold(model)                             │
│                                                                   │
│    ┌──────────────┐  ┌──────────────────┐  ┌──────────────────────┐  │
│    │ < Threshold  │  │ >= Threshold &   │  │ >= Blocking limit    │  │
│    │ Normal API call │  │ < Blocking       │  │ Block input          │  │
```

```
│    └──────────────┘  └────────────────────┘  └──────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────┘
    │
    ▼ (threshold exceeded)
┌──────────────────────────────────────────────────────────────────────────┐
│ 4. Session Memory Compact / Auto-Compact                                 │
│    autoCompactIfNeeded(messages, toolUseContext, ...)                    │
│                                                                          │
│    ┌─────────────────┐    ┌──────────────────────────────────┐          │
│    │ SM-Compact?     │    │ Traditional Compaction           │          │
│    │ (priority use)  │    │ compactConversation()            │          │
│    │ No API call     │    │ ├─ fork sub-agent generate summary │        │
│    └─────────────────┘    │ ├─ clean file read cache         │          │
│                           │ ├─ re-inject files/Skills/Plan    │          │
│                           │ ├─ create compaction boundary     │          │
│                           │ └─ PostCompact Hooks + cleanup    │          │
│                           └──────────────────────────────────┘          │
│                                                                          │
│    Success → Reset circuit breaker consecutiveFailures = 0               │
│    Failure → consecutiveFailures++ → 3 times, circuit breaks             │
└──────────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ 5. API Call (queryModelWithStreaming)                                    │
│    ├─ normalizeMessagesForAPI()                                          │
│    ├─ inject cache_reference + cache_edits (if any)                      │
│    ├─ Send anthropic.beta.messages.create streaming request              │
│    └─ Process streaming response                                         │
└──────────────────────────────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│ 6. Stop Hooks (background tasks, parallel execution)                     │
│                                                                          │
│    ┌──────────────────────┐  ┌──────────────────────────────────────┐   │
│    │ extractMemories      │  │ extractSessionMemory (sessionMemory) │   │
│    │ → Persistent memory  │  │ → Update session memory Markdown     │   │
│    │ → memory/ dir        │  │ → ~/.claude/sessions/<id>/           │   │
│    └──────────────────────┘  └──────────────────────────────────────┘   │
│                                                                          │
│    Mutually exclusive: If main agent already wrote to memory, skip extraction │
└──────────────────────────────────────────────────────────────────────────┘
    │
    ▼
  User sees response
```

19.2 Compaction Message Construction Flow

```
CompactionResult
    │
    ├─ boundaryMarker        ← SystemMessage(type=compact_boundary)
    │       └─ compactMetadata: { trigger, preTokens, preservedSegment }
    │
    ├─ summaryMessages[]      ← UserMessage(isCompactSummary=true)
    │       └─ isVisibleInTranscriptOnly: true (not sent to API)
    │
```

```
    ├─ messagesToKeep[]       ← Most recent N messages (SM-Compact scenario)
    │
    ├─ attachments[]         ← AttachmentMessage[]
    │       ├─ postCompactFileAttachments (most recent reads, max 5, total 50K)
    │       ├─ taskStatus (async agent state)
    │       ├─ plan_file_reference (Plan file)
    │       ├─ plan_mode (Plan mode reminder)
    │       ├─ invoked_skills (used skills, max 5K each)
    │       └─ deferredToolsDelta / agentListingDelta / mcpInstructionsDelta
    │
    └─ hookResults[]          ← HookResultMessage[]
            └─ Results from processSessionStartHooks()
                └─ Reload CLAUDE.md, memory files, etc.

Assembly: buildPostCompactMessages(result)
    [boundaryMarker, ...summaryMessages, ...messagesToKeep, ...attachments, ...hookResults]
```

19.3 Token Consumption by Context Layer

```
┌──────────────────────────────────────────────────────────────────────┐
│                           Model Context Window                        │
│                                                                       │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────┐ ┌──────────────┐ │
│  │ System prompt│ │ Tool definitions│ │User context│ │ Message history│ │
│  │ ~30-50K     │ │ ~5-20K       │ │ ~5K     │ │ Variable    │ │
│  └──────────────┘ └──────────────┘ └──────────┘ └──────────────┘ │
│                                                                       │
│  Message history composition:                                         │
│  ├─ Compaction boundary messages (fixed ~200B)                        │
│  ├─ Summary messages (structured text ~1-5K)                          │
│  ├─ Preserved tail messages (most recent N, 10K-40K)                  │
│  ├─ File attachments (max 5 files, 5K each)                           │
│  ├─ Skills attachments (max 25K total)                                │
│  ├─ Tool results after microcompact cleanup (only most recent few)    │
│  └─ Latest user message + assistant response                          │
└──────────────────────────────────────────────────────────────────────┘
```

Appendix A: Key File Index

File Path	Responsibility Description
`utils/tokens.ts`	Token counting (exact + estimation), API usage extraction
`utils/context.ts`	Context window constants and model-related calculations
`utils/contextAnalysis.ts`	Context statistics analysis, duplicate read detection
`services/tokenEstimation.ts`	Rough token estimation (including special type handling)
`services/compact/microCompact.ts`	Microcompact entry, time-triggered + cache editing
`services/compact/timeBasedMCConfig.ts`	Time-based MC GrowthBook configuration
`services/compact/apiMicrocompact.ts`	API layer context management strategies (clear_tool_uses, clear_thinking)
`services/compact/autoCompact.ts`	Auto-compact trigger, circuit breaker, threshold system
`services/compact/compact.ts`	Full compaction core logic (summary generation, file recovery)
`services/compact/prompt.ts`	Compaction summary prompt template
`services/compact/grouping.ts`	API round message grouping (by message.id)

File Path	Responsibility Description
`services/compact/postCompactCleanup.ts`	Post-compaction cache cleanup
`services/compact/compactWarningState.ts`	Compaction warning state management
`services/compact/compactWarningHook.ts`	React hook: subscribe to warning suppression state
`services/compact/sessionMemoryCompact.ts`	Session memory compaction (SM-Compact)
`services/SessionMemory/sessionMemory.ts`	Session memory extraction and update
`services/SessionMemory/sessionMemoryUtils.ts`	Session memory utilities, threshold calculation
`services/SessionMemory/prompts.ts`	Session memory template and prompt generation
`services/extractMemories/extractMemories.ts`	Persistent memory extraction
`context.ts`	System context (Git status), user context (CLAUDE.md)
`history.ts`	REPL prompt history management
`utils/messages.ts`	Message normalization, boundary messages, utilities
`utils/sessionStorage.ts`	Session message persistence
`services/contextCollapse/index.js`	Context Collapse (experimental, dynamic load)

Appendix B: All Configurable Constants

Constant	Value	File
`MODEL_CONTEXT_WINDOW_DEFAULT`	200,000	`utils/context.ts`
`COMPACT_MAX_OUTPUT_TOKENS`	20,000	`utils/context.ts`
`AUTOCOMPACT_BUFFER_TOKENS`	13,000	`services/compact/autoCompact.ts`
`WARNING_THRESHOLD_BUFFER_TOKENS`	20,000	`services/compact/autoCompact.ts`
`MANUAL_COMPACT_BUFFER_TOKENS`	3,000	`services/compact/autoCompact.ts`
`MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES`	3	`services/compact/autoCompact.ts`
`POST_COMPACT_MAX_FILES_TO_RESTORE`	5	`services/compact/compact.ts`
`POST_COMPACT_TOKEN_BUDGET`	50,000	`services/compact/compact.ts`
`POST_COMPACT_MAX_TOKENS_PER_FILE`	5,000	`services/compact/compact.ts`
`POST_COMPACT_MAX_TOKENS_PER_SKILL`	5,000	`services/compact/compact.ts`
`POST_COMPACT_SKILLS_TOKEN_BUDGET`	25,000	`services/compact/compact.ts`
`MAX_PTL_RETRIES`	3	`services/compact/compact.ts`
`MAX_SECTION_LENGTH`	2,000	`services/SessionMemory/prompts.ts`
`MAX_TOTAL_SESSION_MEMORY_TOKENS`	12,000	`services/SessionMemory/prompts.ts`
`DEFAULT_SM_COMPACT_CONFIG.minTokens`	10,000	`services/compact/sessionMemoryCompact.ts`
`DEFAULT_SM_COMPACT_CONFIG.maxTokens`	40,000	`services/compact/sessionMemoryCompact.ts`
`MINIMUM_MESSAGETOKENS_TO_INIT`	10,000	`services/SessionMemory/sessionMemoryUtils.ts`
`MINIMUM_TOKENS_BETWEEN_UPDATE`	5,000	`services/SessionMemory/sessionMemoryUtils.ts`
`TOOL_CALLS_BETWEEN_UPDATES`	3	`services/SessionMemory/sessionMemoryUtils.ts`
`TIME_BASED_MC_CONFIG.gapThresholdMinutes`	60	`services/compact/timeBasedMCConfig.ts`
`DEFAULT_MAX_INPUT_TOKENS`	180,000	`services/compact/apiMicrocompact.ts`
`DEFAULT_TARGET_INPUT_TOKENS`	40,000	`services/compact/apiMicrocompact.ts`
`IMAGE_MAX_TOKEN_SIZE`	2,000	`services/compact/microCompact.ts`
`MAX_HISTORY_ITEMS`	100	`history.ts`
`MAX_STATUS_CHARS`	2,000	`context.ts`
`TOKEN_COUNT_THINKING_BUDGET`	1,024	`services/tokenEstimation.ts`